

Testing and Inspecting to Ensure High Quality

Adapted from :

Timothy Lethbridge and Robert Laganier,

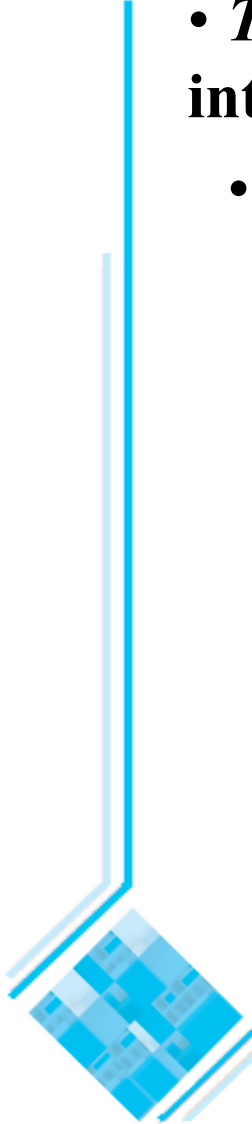
Object-Oriented Software Engineering –

Practical Software Development using UML and Java, 2005

(chapter 10)

10.1 Basic definitions

- A *failure* is an unacceptable behaviour exhibited by a system
 - Any violation of a *functional* or a *non-functional* (*explicit* or *implicit*) requirement should be considered a failure.
 - Examples:
 - » an outright crash,
 - » the production of incorrect output,
 - » the system running too slowly,
 - » the user having trouble finding online help, etc.
 - The frequency of failures measures the *reliability*.
 - Important objective: maximize reliability $\Leftrightarrow$ minimize the failure rate
- A *defect* is a flaw in any aspect of the system that contributes, or may potentially contribute, to the occurrence of one or more failures
 - could be in the requirements, the design or the code
 - It might take several defects to cause a particular failure
- An *error* is an omission or an inappropriate decision by a software developer that leads to the introduction of a defect

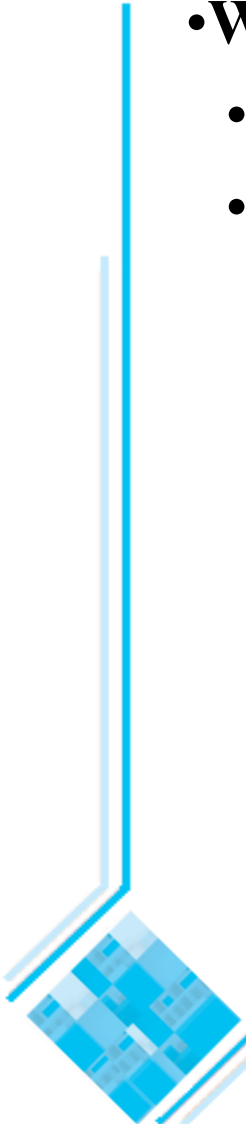
- 
- **Testing** is the process of executing a program with the intent of finding any defect that might be present.
 - We emphasize that
 - program testing can only demonstrate the presence of defects not their absence!*

10.2 Effective and efficient testing

- To test *effectively*, you must use a strategy that uncovers as many defects as possible
- To test *efficiently*, you must find the largest possible number of defects using the fewest possible tests
 - Testing is like detective work:
 - The tester must try to understand how programmers and designers think, so as to better find defects.
 - The tester must not leave anything uncovered, and must be suspicious of everything.
 - It does not pay to take an excessive amount of time; the tester has to be *efficient*.

Basic testing techniques

- **We consider two basic testing techniques:**
 - Path testing (applicable in ‘white-box’ testing)
 - Equivalence partitioning (applicable in both ‘white-box’ and ‘black-box’ testing)



White-box testing

Also called ‘glass-box’ or ‘structural’ testing

Testers have access to the system design

- They can
 - Examine the design documents
 - View the code
 - Observe at run time the steps taken by algorithms and their internal data
- Individual programmers often informally employ white-box testing to verify their own code

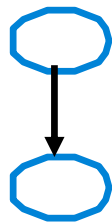
Control flow graph for white-box testing

- A well-known structural testing technique is *path testing*.
 - Path testing is effective at method (procedure, function) level and it is based on the control flow graph (CFG) of the method.
- It helps the programmer to systematically test the code
 - Each branch in the code (such as an **if** or a **while** statement) creates a node in the CFG
 - The testing strategy has to reach a targeted coverage of statements and branches. The objective can be to design tests for:
 - covering all possible paths (most often unfeasible)
 - covering all possible edges** (each statement is executed at least once)
 - The **cyclomatic complexity** (cc) of the CFG gives the maximum number of independent paths that must be tested in order to cover all possible edges [MCC76]:
 - » $cc(CFG) = noEdges - noNodes + 2$
 $= noBinaryDecisionPredicates + 1$
 - It is preferable to limit cc to 10 (11-20 moderate risk, 21-50 high risk, > 50 untestable / very high risk)

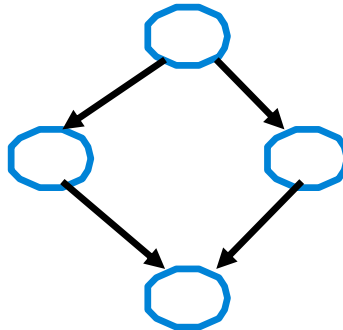
Control flow graph for white-box testing

- **The CFG of a structured program can be built as a combination of CFGs for the basic structured constructs.**

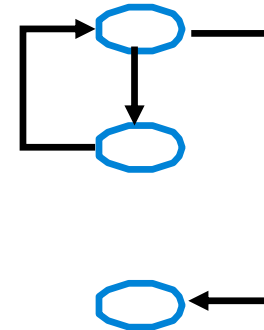
Sequence



If



While



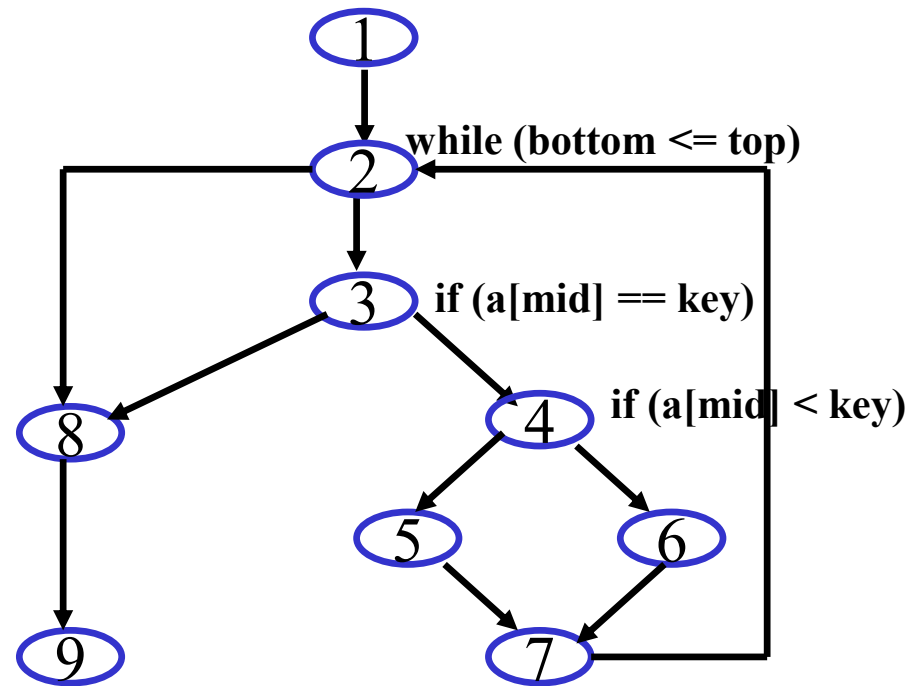
- **The CFG**

- is a directed graph
- with **single entry & single exit points**

Example: CFG of a binary search routine ([Som00], ch. 20)

```
class BinSearch {  
    ...  
    public static void search (int key, int[] a, Rez r)  
    {  
        int mid; int bottom = 0;  
        int top = a.length - 1;  
        r.found = false; r.index = -1;  
        while (bottom <= top) {  
            mid = (top + bottom) / 2;  
            if (a[mid] == key) {  
                r.index = mid;  
                r.found = true;  
                return;  
            } else {  
                if (a[mid] < key)  
                    bottom = mid + 1;  
                else  
                    top = mid - 1;  
            }  
        }  
    }  
}
```

cc(CFG)
= noEdges - noNodes + 2
= noBinaryDecisionPredicates + 1
= 4



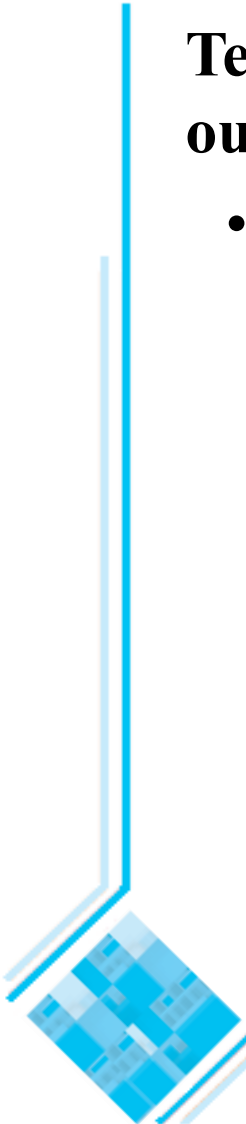
A set of independent paths:

- 1,2,8,9
- 1,2,3,8,9
- 1,2,3,4,5,7,2,8,9
- 1,2,3,4,6,7,2,8,9

Black-box testing

Testers provide the system with inputs and observe the outputs

- They can see none of:
 - The source code
 - The internal data
 - Any of the design documentation describing the system's internals

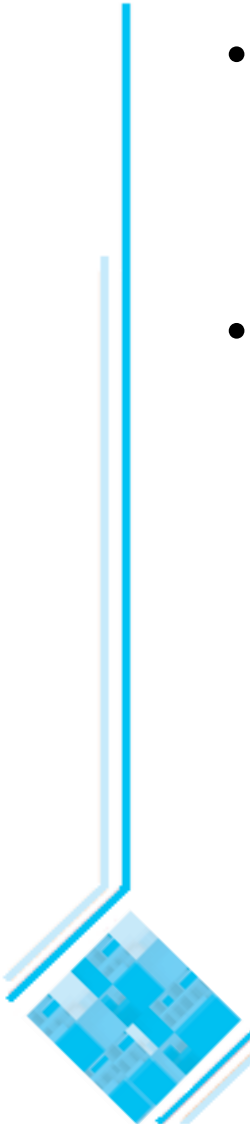


Equivalence classes

- It is inappropriate to test by *brute force*, using *every possible* input value
 - Takes a huge amount of time
 - Is impractical
 - Is pointless!
- You should divide the possible inputs into groups which you believe will be treated similarly by all algorithms.
 - Such a group is called an **equivalence class (EC)**
 - A tester needs only to run **one test per EC**
 - The tester has to
 - understand the required input,
 - appreciate how the software may have been designed

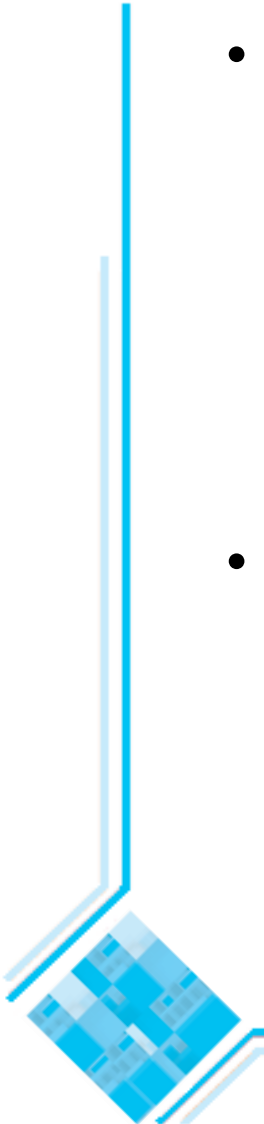
Examples of equivalence classes

- Valid input is a month number (1-12)
 - 3 ECs: $[-\infty..0]$, $[1..12]$, $[13..\infty]$
- Valid input is one of ten (10) strings representing a type of fuel
 - 11 ECs:
 - 10 classes, one for each string
 - 1 class representing all other strings



Combinations of equivalence classes

- **Combinatorial explosion** means that you cannot realistically test every possible system-wide EC.
 - For example, if there are 4 different inputs, each with 5 ECs, then there are $5 \times 5 \times 5 \times 5 = 5^4 (=625)$ possible system-wide ECs.
 - It is impractical to design 625 different test cases.
- In practice, the number of tests must be kept in reasonable limits !



Example equivalence class combinations

- **Example: a system that contains information about all kinds of land vehicles (e.g. passenger, racing, etc). Five types of input:**
 1. Type of unit: 'Metric' (km/h), or 'Imperial or US' (mph)
 - By using 'radio buttons' there is no possibility of an invalid input
 - 2 ECs: Metric, US/Imperial
 2. Maximum speed (integer): 1-750 km/h, or 1-500 mph
 - Validity depends on the type of unit (metric or US/Imperial)
 - 4 ECs: $[-\infty..0]$, $[1..500]$, $[501..750]$, $[751.. \infty]$
 3. Type of fuel (string): one of 10 possible strings
 - 11 ECs (10 plus 1 class for any invalid string)
 4. Time to accelerate to 100km/h or 60 mph (integer): 1-100s
 - 3 ECs: $[-\infty..0]$, $[1..100]$, $[101..\infty]$
 5. Range (integer): 1-5000 km, or 1-3000 miles.
 - Validity depends on the the type of unit (metric or US/Imperial)
 - 4 ECs: $[-\infty..0]$, $[1..3000]$, $[3001..5000]$, $[5001.. \infty]$

Example equivalence class combinations

- Total number of ECs: $2 \times 4 \times 11 \times 3 \times 4 = 1056$ (!!)
- A reasonable strategy for reducing the number of tests:
 1. You should first make sure that at least one test is run with every EC of every individual input.
 2. You should test all combinations where an input is likely to *affect the interpretation* of another.
 - You should also test a few other random combinations of ECs.
- By using this strategy the number of tests for the vehicles example can be reduced from 1056 to 11:
 - You run one test for each EC of input 3 (11 tests)
 - While doing this you vary (in parallel)
 - the 8 possible combinations of ECs of inputs 1 and 2
 - the 8 possible combinations of ECs of inputs 1 and 5
 - the 3 possible ECs of input 4

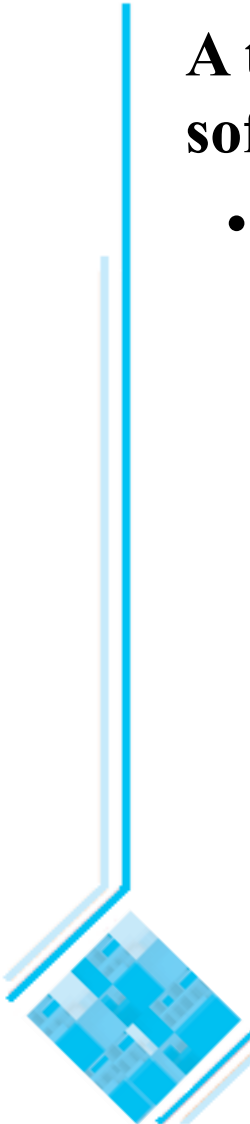
Testing at the boundaries of equivalence classes

- More errors in software occur at **the boundaries of equivalence classes**
- The idea of ECs testing should be expanded to specifically test values at the extremes of each EC.
 - E.g. The number 0 often causes problems
- *E.g.:* If the valid input is a month number (1-12)
 - Test the ECs as before
 - Test 0, 1, 12 and 13 as well as very large positive and negative values

Detecting specific categories of defects

A tester must try to uncover any defects the other software engineers might have introduced.

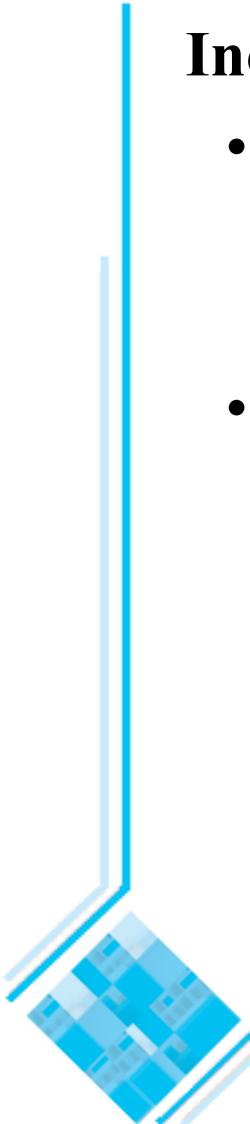
- This means designing tests that explicitly try to catch a range of specific types of defects that commonly occur



10.3 Defects in Ordinary Algorithms

Incorrect logical conditions

- *Defect:*
 - The logical conditions that govern looping and if-then-else statements are wrongly formulated.
- *Testing strategy:*
 - Use equivalence class and boundary testing.
 - Consider as an input each variable used in a rule or logical condition.



Example of incorrect logical conditions defect (in software for an aircraft landing gear system)

- **Specification:**

—The landing gear must be deployed whenever the plane is within 2 minutes from landing or takeoff, or within 2000 feet from the ground. If visibility is less than 1000 feet, then the landing gear must be deployed whenever the plane is within 3 minutes from landing or lower than 2500 feet.

- (Otherwise, the aircraft's **alarm** is supposed to sound !)

- What is the **hard-to-find defect** in the following **implementation** ?

```
if (!landingGearDeployed &&
    (min(now-takeoffTime, estLandTime-now) <
      (visibility < 1000 ? 180 : 120) ||
      relativeAltitude <
        (visibility < 1000 ? 2500 : 2000)
    )
{
    throw
        new LandingGearException();
}
```

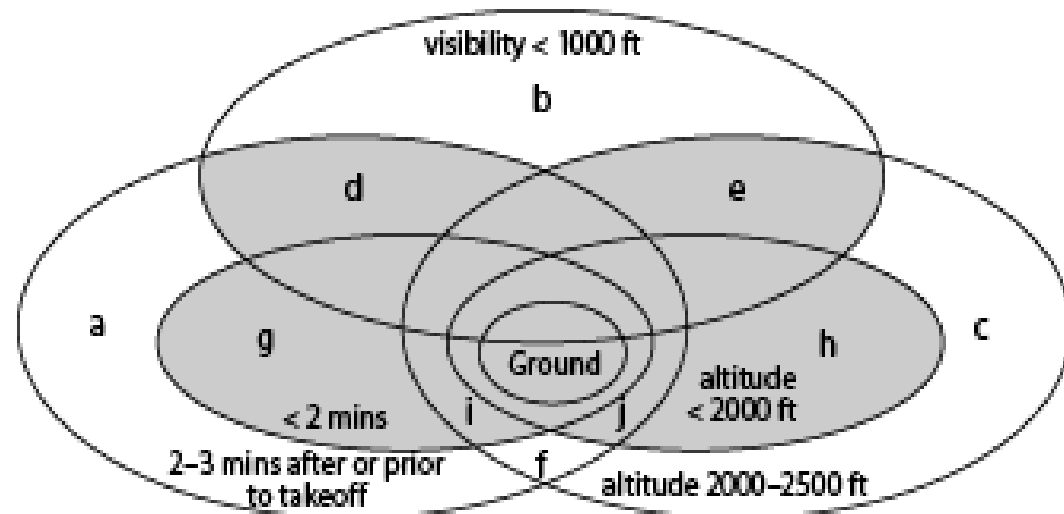
Example of incorrect logical conditions defect

—Total number of system ECs: 108

—For such a **critical system** all the 108 ECs (plus their boundaries) should be tested (in a simulated environment).

—Venn diagram: the gray area indicates the ECs where the alarm should sound if the landing gear is not deployed.

<i>Variable affecting condition</i>	<i>Equivalence classes</i>
Time since take-off	3: Within 2 minutes after take-off, 2–3 minutes after take-off, more than 3 minutes after takeoff
Time to landing	3: Within 2 minutes prior to landing, 2–3 minutes prior to landing, more than 3 minutes prior to landing
Relative altitude	3: < 2000 feet, 2000 feet to 2500 feet, 2500 feet
Visibility	2: < 1000 feet, 1000 feet
Landing gear deployed	2: true, false



www.lloseng.com

Defects in Ordinary Algorithms

Not setting up the correct preconditions for an algorithm

- *Defect:*
 - *Preconditions* state what must be true before the algorithm should be executed.
 - A defect would exist if a program proceeds to do its work, even when the preconditions are not satisfied.
- *Testing strategy:*
 - Run test cases in which each precondition is not satisfied.

Defects in Ordinary Algorithms

Not handling null conditions

- *Defect:*
 - A *null condition* is a situation where there normally are one or more data items to process, but sometimes there are none.
 - It is a defect when a program behaves abnormally when a null condition is encountered.
- *Testing strategy:*
 - Determine the null conditions and run appropriate tests.
- *Example:*
 - A program that calculates the average sales of the members of each division in an organization.
 - Null condition: a division that has no members
 - Typical defect: the program attempts to divide by zero (zero sales divided by zero members)
 - Test the behavior of the program for a division with zero members.

Defects in Ordinary Algorithms

Not handling singleton or non-singleton conditions

- *Defect:*
 - A *singleton condition* is similar to a null condition. It occurs when there is normally *more than one* of something, but sometimes there is only one.
 - A *non-singleton condition* is the inverse (there is normally exactly *one*, but occasionally there is more than one).
 - Defects occur when the unusual case is not properly handled.
- *Testing strategy:*
 - Determine unusual conditions and run appropriate tests.



Defects in Ordinary Algorithms

Off-by-one errors

- *Defect:*
 - A program loops one too many times or one too few times.
 - Or a program inappropriately adds or subtracts one
 - E.g. by using ++v instead of v++ (or vice versa)
 - This is a particularly common type of defect.
- *Testing strategy:*
 - Develop **boundary tests** in which you verify that the program:
 - performs the correct number of iterations
 - computes the correct numerical answer

Defects in Ordinary Algorithms

- *Example:* Assuming Java-like 0-based indexing, the following loop always skips the first element:

```
for (i=1; i < arrayname.length; i++)  
{  
    /* do something */  
}
```

- The problem can easily be avoided by the designer by using the concept of an Iterator (in Java the `Iterator` class).
- But the tester must assume the worst and design boundary tests.

Defects in Ordinary Algorithms

Not terminating a loop or recursion

- *Defect:*

- A loop or a recursion does not always terminate, i.e. in some conditions it is ‘infinite’.

- It is the responsibility of the programmer to avoid (undesirable) infinite behaviour, but the tester must assume the worst and design appropriate test cases.

- *Testing strategies:*

- Analyse what causes a repetitive action to be stopped.

- Run test cases that you anticipate might not be handled correctly.

Defects in Ordinary Algorithms

Use of inappropriate standard algorithms

- *Defect:*
 - An inappropriate standard algorithm is one that is unnecessarily inefficient or has some other property that is widely recognized as being bad.
- *Testing strategies:*
 - The tester has to know the properties of algorithms and design tests that will determine whether any undesirable algorithms have been implemented.
- *Example:* The designer uses a ‘bubble sort’ instead of ‘quick sort’.
 - The tester can increase (e.g. *double*) the number of items being sorted and observe how execution time is affected.

10.4 Defects in Numerical Algorithms

- **Not using enough bits or digits to store maximum values**
 - *Testing strategies:* Test using very large numbers
 - *Example:* In 5 June 1996 the first test flight of the French Ariane-5 Launcher ended in failure.
 - The failure was caused by a *software error*: an attempt to convert a (too large) 64-bit floating point number to a 16-bit signed integer.
 - It was one of the most expensive computer bugs in history (causing a loss of approx. \$500 million).



Defects in Numerical Algorithms

Assuming a floating point value will be exactly equal to some other value

- *Defect:*

- If you perform an arithmetic calculation on a floating point value, then the result will very rarely be computed exactly.
- It is a defect to test if a floating point value is exactly equal to some other value.
- To test equality, you should always test if it is within a small range around that value.

- *Testing strategies:*

- Standard **boundary testing** should detect this type of defect.

Defects in Numerical Algorithms

Example of defect in testing floating value equality:

Bad:

```
for (double d = 0.0; d != 10.0; d+=2.0) { ... }
```

Better:

```
for (double d = 0.0; d < 10.0; d+=2.0) { ... }
```

10.5 Defects in Timing and Co-ordination

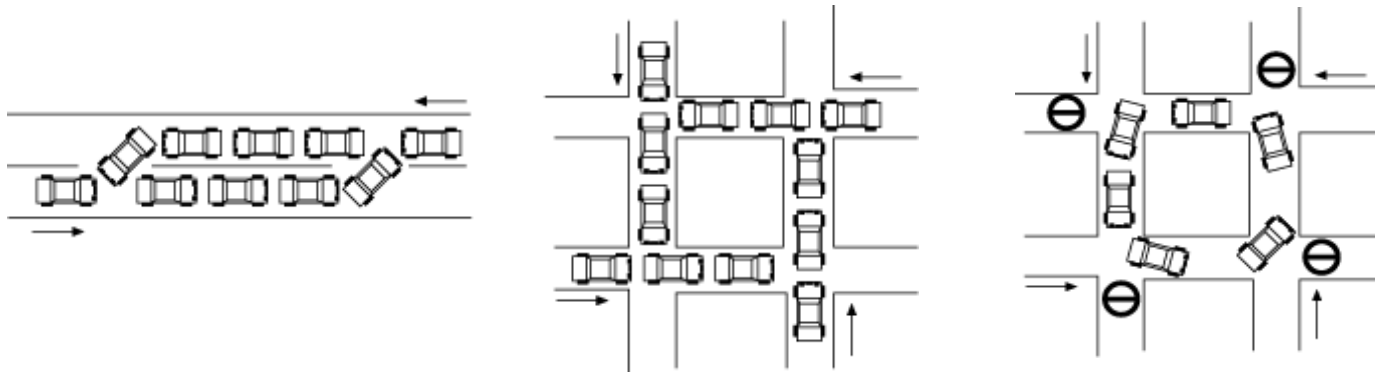
Deadlock and livelock

- *Defects:*

- A *deadlock* is a situation where two or more threads are stopped, waiting for each other to do something before either can proceed.

- The system is hung

- Livelock* is similar, but now the system can do some computations, but can never get out of some states.



Defects in Timing and Co-ordination

Deadlock and livelock

- *Testing strategies:*

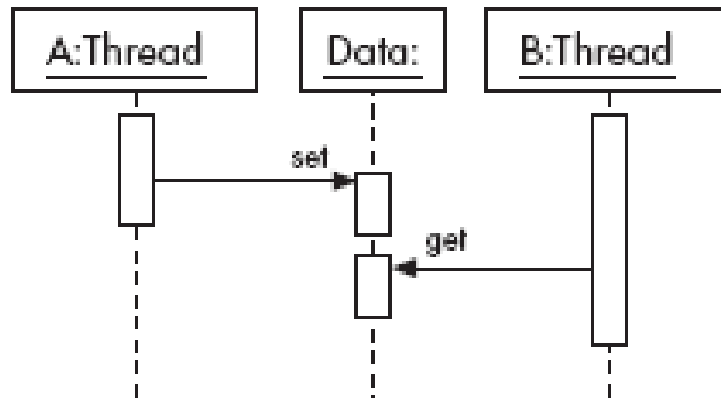
- Deadlocks and livelocks occur due to unusual combinations of conditions that are hard to anticipate or reproduce.
- If black box testing is the only possibility you can:
 - Run a large number of threads concurrently.
 - Deliberately deny resources to one or more threads (e.g. by cutting network connections, or by locking input files).
 - Vary the relative speed of the different threads
- However, the deadlock problem is difficult. Expensive white-box testing and *inspections* may be necessary.
 - *Formal verification techniques* have been successful in producing concurrent systems that have a high level of reliability.

Defects in Timing and Co-ordination

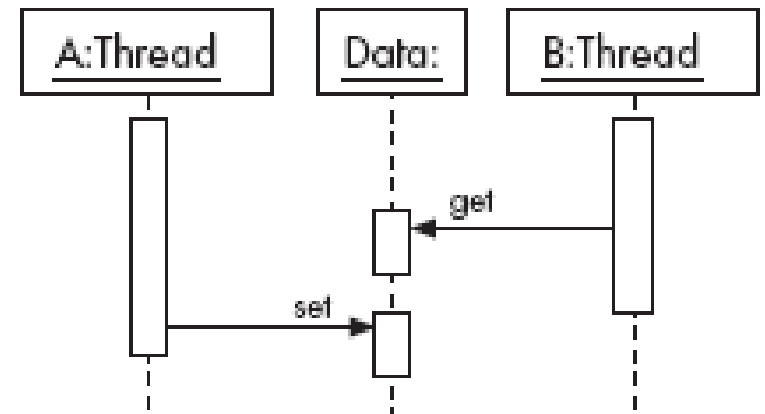
Critical races

- *Defect:*
 - One thread experiences a failure because another thread interferes with the ‘normal’ (or ‘expected’) sequence of events. The defect is not that the other thread tries to do something, but that the program allows the (‘abnormal’) interference to occur.
- *Testing strategies:*
 - Same as for deadlock and livelock.
 - *Testing for deadlock or critical races is generally ineffective, since they are produced by particular chains of events.*
 - It is particularly hard to test for critical races by using black-box testing alone.
 - Once again, *inspection* is often a better solution.

Example of critical race



a) Normal

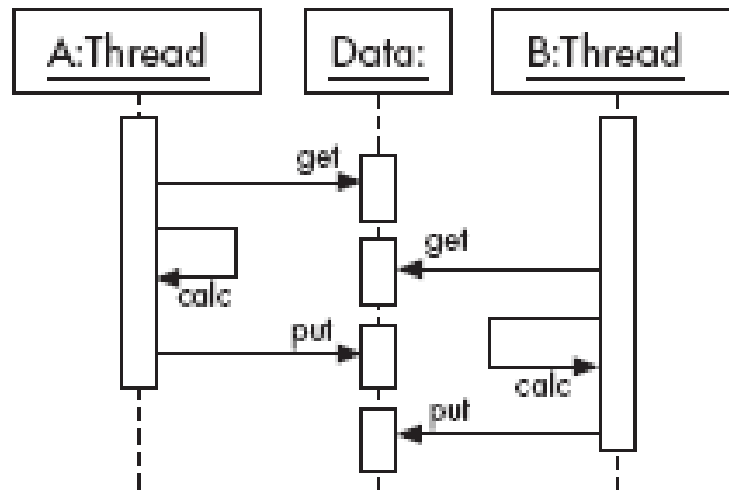


b) Abnormal due to delay in thread A

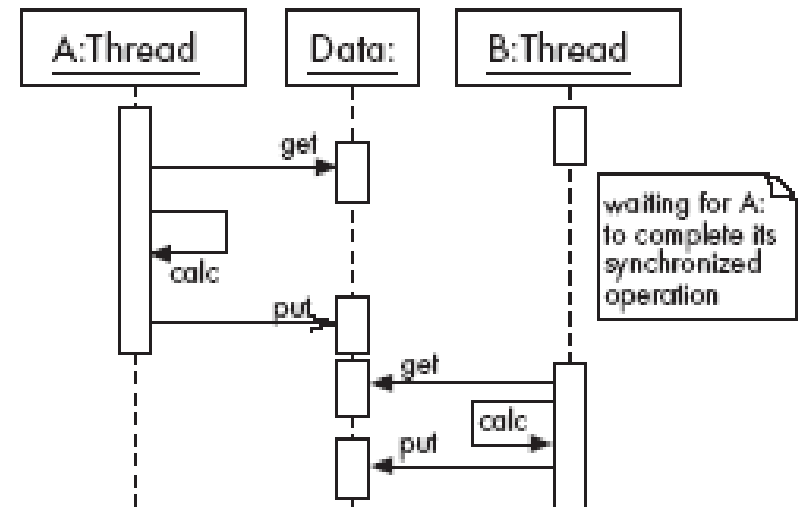
Synchronization

- **The easiest way to deal with deadlocks and critical races is to avoid them at design time !**
- Critical races can be prevented by *locking* data so that they cannot be accessed by other threads when they are not ready.
 - In Java, the **synchronized** keyword can be used.
 - It ensures that no other thread can access an object until the synchronized method terminates.

Example of a synchronized method



a) Abnormal: The value put by thread A is immediately overwritten by the value put by thread B.



b) The problem has been solved by accessing the data using synchronized methods.

10.9 Strategies for Testing Large Systems

Integration testing: big bang versus incremental testing

- In *big bang* testing, you take the entire system and test it all at once.
 - Impractical for most nontrivial systems.
 - Errors resulting from modules interactions are difficult to locate.
- A better strategy in most cases is *incremental* testing.
 - When you encounter a failure you can *locate* the defect more easily.
 - Unfortunately, there is a high probability that efforts to remove defects will actually add new ones. This phenomenon is called **the ripple effect**.
 - After a defect is located and fixed you may have to perform regression testing.
 - **Regression testing** is the process of retesting (parts of) a system that was previously built and tested when new functionality is added.

Strategies for Testing Large Systems

Incremental testing can be performed *horizontally* or *vertically*.

- Horizontal testing can only be applied when the system is divided into *separate* subsystems.
- The basic strategies for *vertical incremental testing* are:

—Top-down

- In this approach you have to design **stubs** (i.e. modules with the same interface but very limited functionality) that simulate lower-level modules.

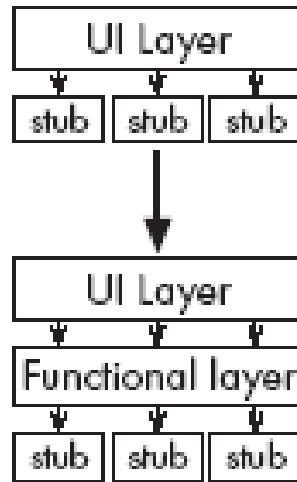
—Bottom-up

- In this approach you have to design **test drivers** for the lower-level modules.
 - » A test driver is a module designed specifically for testing by making calls to the lower layers.

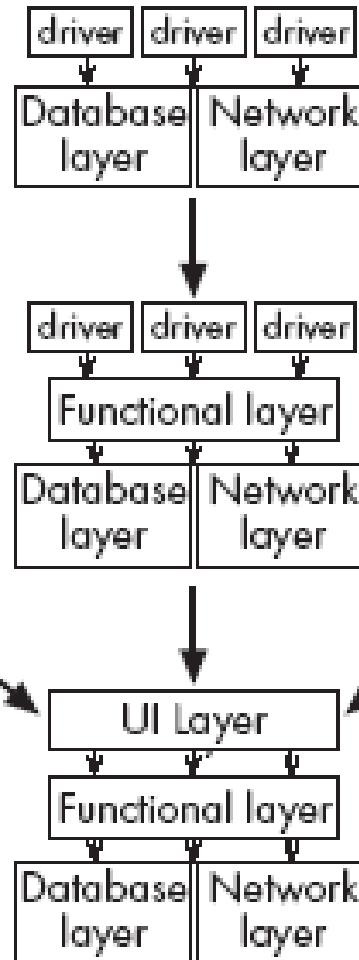
—Sandwich

Vertical strategies for incremental integration testing

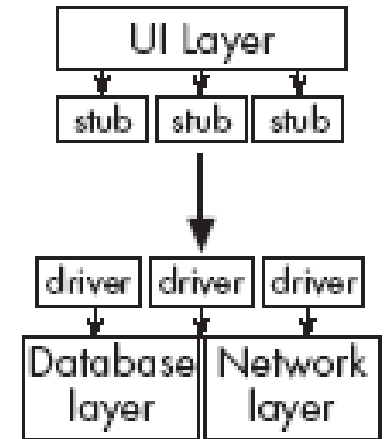
Top-down testing



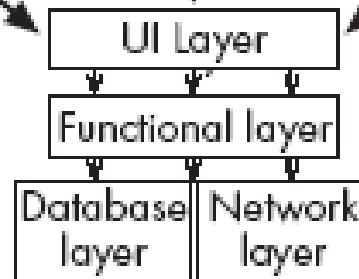
Bottom-up testing



Sandwich testing



Fully
integrated
system



- Most of the testing techniques and strategies given in this chapter work just as well for OO software as for non-OO software.
- In an OO system method and class testing correspond to *unit testing*.
 - An OO class is a unit of protection.
 - You can write special methods within each class to act as test drivers for the methods of that class.
 - In Java, you can create a main() method (to act as a test driver) in each class in order to exercise the other methods.
 - JUnit [Bec04] is a professional unit testing tool for Java classes:
<http://www.junit.org>
- However, in an OO system there is no obvious ‘top’ to the system.
 - Neither top-down, nor bottom-up integration are really appropriate for OO systems [Som00].



Use case integration testing

- **Use-case or scenario-based testing** is often the most effective *integration testing strategy* for OO systems [JBR99,Som00].
 - Black-box tests can be derived directly from the use case model.
 - Each use case can give rise to a number of *test cases*.
 - White-box tests are based on the use case realizations in the design model.
 - Interaction diagrams (e.g. *sequence diagrams*) provide essential information for test cases design.

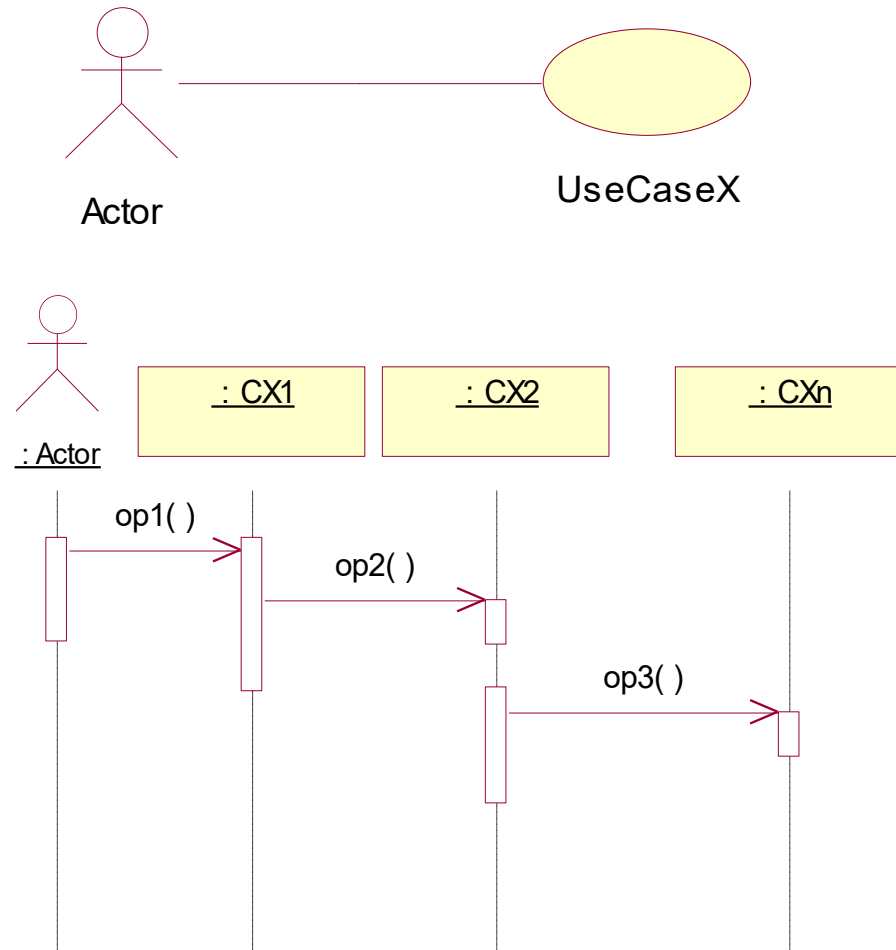


Test Cases

- A *test case* is an explicit set of instructions designed to detect a particular class of defect in a software system.
- Each test case should include the following information:
 - what to test (identification information),
 - with which input,
 - (expected) result,
 - and under which conditions.
- The specification may also include
 - detailed instructions on how to perform the test case (the *test procedure*)
 - cleanup instructions (when needed)
 - Example: instructions to remove erroneous test data (added during the execution of the test case) from some external database.

Designing integration test cases based on sequence diagrams

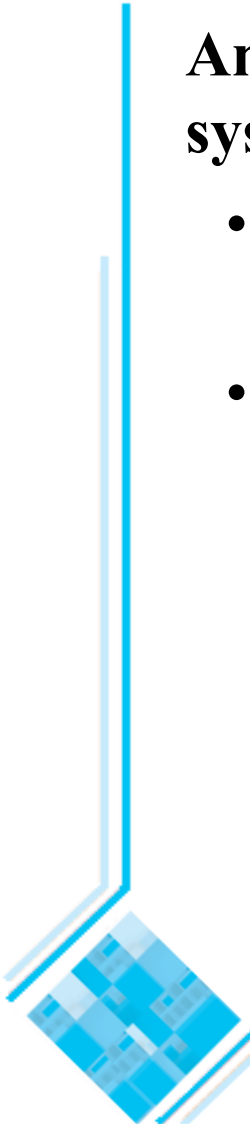
- Assume that:
 - CX1, CX2, ..., CXn are the classes that participate the design of UseCaseX.
 - The **sequence diagram** describes a use scenario for UseCaseX.
- Based on such an interaction (sequence) diagram, you can design an **integration test case** by specifying the conditions (start state, input from the actor, etc) that make the sequence happen [JBR99].
 - A **defect** is identified if the actual interaction (captured during the execution of the test case) and the interaction specified by the sequence diagram are not the same.
- The **actual interaction** can be captured
 - by running a debugger
 - by using print statements (e.g. in Java `System.out.println()`), etc.



10.10 Inspections

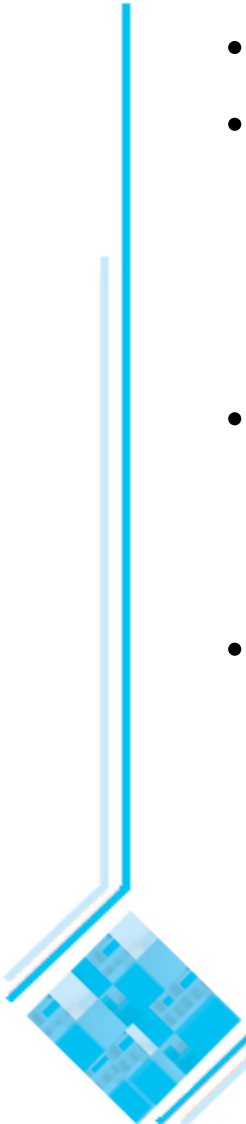
An inspection is an activity in which one or more people systematically

- Examine source code or documentation, looking for defects.
- Normally, inspection involves a meeting...
 - Although participants can also inspect alone at their desks.



Roles on inspection teams

- The *author*
- The *moderator*.
 - Calls and runs the meeting.
 - Makes sure that the general principles of inspection are adhered to.
- The *secretary*.
 - Responsible for recording the defects when they are found.
 - Must have a thorough knowledge of software engineering.
- *Paraphrasers*.
 - Step through the document explaining it in their own words.

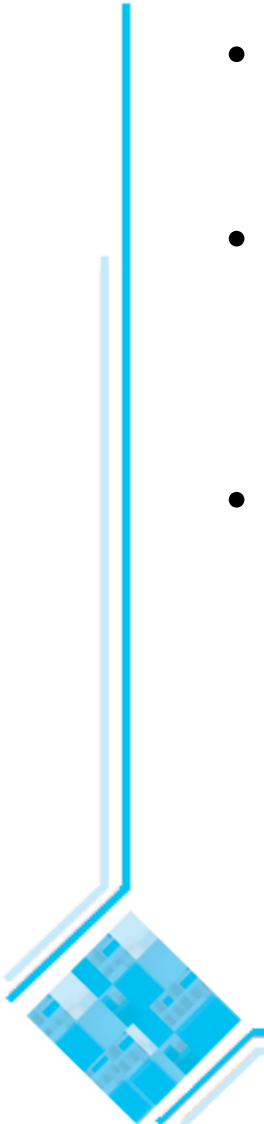


Principles of inspecting

- Inspect the most important documents of all types
 - code, design documents, test plans and requirements
- Choose an effective and efficient inspection team
 - between two and five people
 - Including experienced software engineers
- Require that participants prepare for inspections
 - They should study the documents prior to the meeting and come prepared with a list of defects
- Only inspect documents that are ready
 - Attempting to inspect a very poor document will result in defects being missed

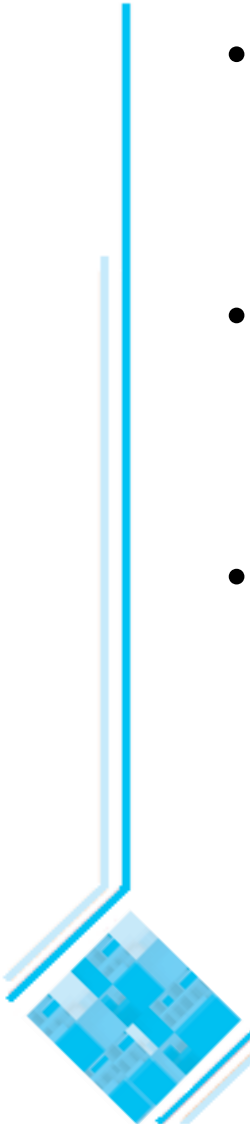
Principles of inspecting

- Avoid discussing how to fix defects
 - Fixing defects can be left to the author
- Avoid discussing style issues
 - Issues like are important, but should be discussed separately
- Do not rush the inspection process
 - A good speed to inspect is
 - 200 lines of code per hour (including comments)
 - or ten pages of text per hour



Principles of inspecting

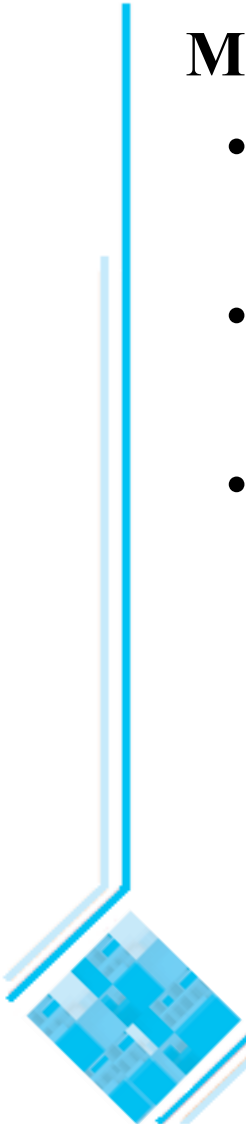
- Avoid making participants tired
 - It is best not to inspect for more than two hours at a time, or for more than four hours a day
- Keep and use logs of inspections
 - You can also use the logs to track the quality of the design process
- Re-inspect when changes are made
 - You should re-inspect any document or code that is changed more than 20%



A peer-review process

Managers are normally not involved

- This allows the participants to express their criticisms more openly, not fearing repercussions
- The members of an inspection team should feel they are all working together to create a better document
- Nobody should be blamed

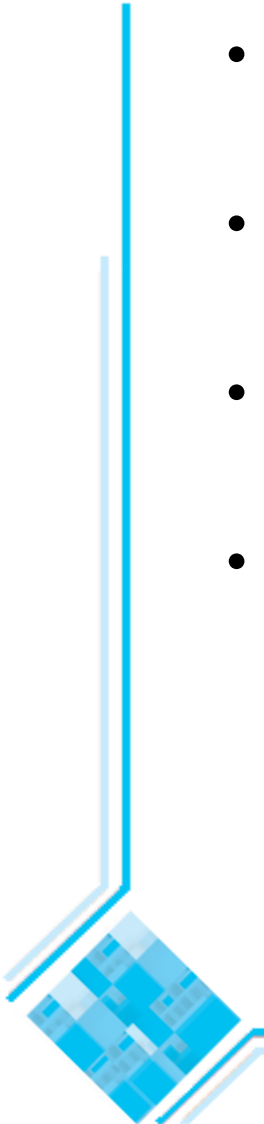


Conducting an inspection meeting

1. The moderator calls the meeting and distributes the documents.
2. The participants prepare for the meeting in advance.
3. At the start of the meeting, the moderator explains the procedures and verifies that everybody has prepared.
4. Paraphraser take turns explaining the contents of the document or code, without reading it verbatim.
 - Requiring that the paraphraser not be the author ensures that the paraphraser say what he or she *sees*, not what the author *intended* to say.
5. Everybody speaks up when they notice a defect.

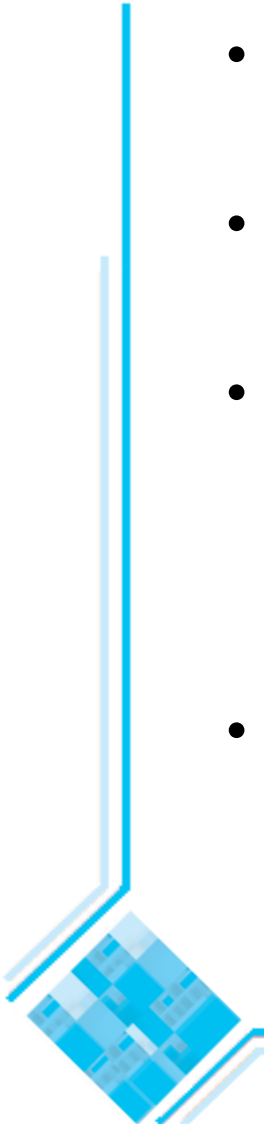
Inspecting compared to testing

- Both testing and inspection rely on different aspects of human intelligence.
- Testing can find defects whose consequences are obvious but which are buried in complex code.
- Inspecting can find defects that relate to maintainability or efficiency.
- The chances of mistakes are reduced if both activities are performed.



Testing or inspecting, which comes first?

- It is important to inspect software *before* extensively testing it.
- The reason for this is that inspecting allows you to quickly get rid of many defects.
- If you test first, and inspectors recommend that redesign is needed, the testing work has been wasted.
 - There is a growing consensus that it is most efficient to inspect software *before* any testing is done.
- Even before developer testing



Additional references

- [Bec04] K. Beck. *JUnit pocket guide*. O'Reilly, 2004.
- [BB99] W. Boggs, M. Boggs. *Mastering UML with Rational Rose* (1st edition), Sybex, 1999.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [MCC76] T. McCabe. A software complexity measure. *IEEE Trans. Software Engineering*, 2:308-320, 1976.
- [Som00] I. Sommerville. *Software Engineering* (6th edition), Addison-Wesley, 2000.
- [Som04] I. Sommerville. *Software Engineering* (7th edition), Addison-Wesley, 2004.